

AEGISAI: An Agentic AI Policy Enforcement Gateway for Customer-Service Compliance

PHASE 1 REPORT

Submitted

By

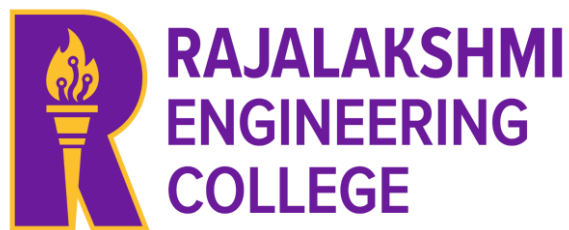
ISHITHA S –	230701117
HEMASHRI U -	230701114
JAYA BHARATHI M -	230701126
ISHWARI RAJMOHAN -	230701118

*In partial fulfilment for the award of the degree
of*

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY, CHENNAI

SEPTEMBER - 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Report titled "**AEGISAI: An Agentic AI Policy Enforcement Gateway for Customer- Service Compliance**" is the bonafide work of **Ishitha S (230701117) , Hemashri U (230701114), Jaya Bharathi M (230701126), Ishwari Rajmohan (230701118)** carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

SIGNATURE

Dr. J. Manoranjini, M.E., Ph.D.,

Professor and Head,

Department of Computer Science and
Engineering,

Rajalakshmi Engineering College,

Chennai-602 105.

SIGNATURE

Mrs. M. Divya, M.E.,

Assistant Professor,

Department of Computer Science and
Engineering,

Rajalakshmi Engineering College,

Chennai-602 105.

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner

External Examiner

ABSTRACT

Customer-support AI agents are now central to e-commerce, travel, finance, and public services, enabling instant responses and automation of routine actions such as refunds, cancellations, and order tracking. However, their autonomous behaviour introduces significant business, legal, and reputational risks when they make unauthorized promises, execute unsafe actions, or leak sensitive data. Recent incidents include chatbots inventing refund policies, agreeing to sell high-value items at symbolic prices due to prompt injection, and exposing customer records. Surveys further indicate that a large majority of enterprises have rolled back or suspended AI customer-service deployments due to hallucinations, data leaks, and lack of auditability.

The proposed project, **AegisAI**, introduces an agentic AI policy enforcement gateway for customer-service compliance. It combines a multi-agent architecture with deterministic business rules and exact tool authorization. A customer-support agent (Agent A), powered by Qwen3-8B via Ollama, generates responses and proposes tool actions such as refunds or order lookups. A Policy Enforcement Gateway evaluates these proposals against customer-service policies covering refund limits, verification, delivery promises, PII protection, and escalation, returning one of four decisions: ALLOW, MODIFY, BLOCK, or ESCALATE. Only approved actions are forwarded to an operations agent (Agent B), and a Tool Interceptor ensures that tool names and arguments match an exact authorization envelope before execution.

Unlike traditional content-level guardrails that focus on toxicity or PII filtering, AegisAI enforces business policies before actions affect customers, controls delegation between agents, and maintains a unified audit chain from request to outcome. The system is implemented using LangGraph for orchestration, FastAPI/Python for the policy engine and interceptor, PostgreSQL for audit storage, and React/TypeScript for dashboards, with a separate mock e-commerce website connected via REST APIs for realistic testing.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavour to put forth this report. Our sincere thanks to our Chairman **Mr. S. MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavouring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. J. MANORANJINI, M.E., Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **Mrs. M. DIVYA, M.E.**, Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project. We thank our Project Coordinator, **Dr. S. MADHUSUDHAN, M.E., Ph.D.**, Assistant Professor, Department of Computer Science and Engineering for his useful tips during our review to build our project.

ISHITHA S: 230701117

HEMASHRI U: 230701114

JAYA BHARATHI M: 230701126

ISHWARI RAJMOHAN: 230701118

TABLE OF CONTENT

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	05
	LIST OF FIGURES	07
	LIST OF ABBREVIATIONS	08
1.	INTRODUCTION	11
	1.1 GENERAL	11
	1.2 OBJECTIVE	12
	1.3 EXISTING SYSTEM	13
	1.4 PROPOSED SYSTEM	13
2.	LITERATURE SURVEY	16
	2.1 LITERATURE REVIEW	16
	2.2 COMPARISON AND DISCUSSION	19
	2.3 CONCLUSION	21
3.	SYSTEM DESIGN	22
	3.1 SYSTEM ARCHITECTURE	22
	3.2 SYSTEM REQUIREMENTS	24
	3.2.1 HARDWARE REQUIREMENTS	24
	3.2.2 SOFTWARE REQUIREMENTS	25
	3.3 SYSTEM DESIGN	26
	3.3.1 ACTIVITY DIAGRAM	26
	3.3.2 DATA FLOW DIAGRAM	27
4.	PROJECT DESCRIPTION	29
	4.1 METHOLOGIES	29
	4.2 CUSTOMER INTERACTION INTAKE AND INPUT GUARD	29
	4.3 CUSTOMER-SUPPORT AGENT (AGENT A)	30
	4.4 AEGISAI POLICY ENFORCEMENT GATEWAY	30
	4.5 POLICY MANAGEMENT MODULE	31

4.6	MULTI-TURN CONTEXT STORE	31
4.7	OPERATIONS AGENT AND TOOL INTERCEPTOR	32
4.8	REMEDICATION AGENT	32
4.9	AUDIT AND INCIDENT STORE	33
4.10	DASHBOARD AND ANALYTICS MODULE	33
5.	IMPLEMENTATION AND RESULT DISCUSSION	35
5.1	IMPLEMENTATION OVERVIEW	35
5.2	DEVELOPMENT ENVIRONMENT AND SETUP	35
5.3	IMPLEMENTATION OF THE POLICY ENFORCEMENT GATEWAY	35
5.4	IMPLEMENTATION OF POLICY DEFINITIONS	38
5.5	IMPLEMENTATION OF THE TOOL INTERCEPTOR AND AUTHORIZATION ENVELOPE	38
5.6	IMPLEMENTATION OF AUDIT LOGGING AND INCIDENT HANDLING	38
5.7	IMPLEMENTATION OF THE DASHBOARD	38
5.8	TESTING	38
6.	CONCLUSION AND FUTURE WORK	39
6.1	CONCLUSION	39
6.2	FUTURE WORK	40
	REFERENCES	41

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 3.1	System Architecture Diagram	23
Figure 3.2	Activity Diagram	27
Figure 3.3	Data Flow Diagram	28

LIST OF ABBREVIATION

Abbreviation	Description
AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
DFD	Data Flow Diagram
DNN	Deep Neural Network
ETA	Expected Time of Arrival
GPU	Graphics Processing Unit
GPS	Global Positioning System
HITL	Human-in-the-Loop
IoT	Internet of Things
JSON	JavaScript Object Notation
LangGraph	Language Graph Orchestration Framework
LLM	Large Language Model
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
ML	Machine Learning
MSE	Mean Square Error
NLP	Natural Language Processing
Ollama	Open Large Language Model Assistant
PII	Personally Identifiable Information

Abbreviation	Description
PostgreSQL	Postgres Structured Query Language
Qwen3-8B	Qwen Version 3 with 8 Billion Parameters
ReLU	Rectified Linear Unit
RNN	Recurrent Neural Network
RMSE	Root Mean Square Error
REST	Representational State Transfer
TN	Tamil Nadu
TPU	Tensor Processing Unit
UI	User Interface
UML	Unified Modeling Language
YAML	Yet Another Markup Language
AegisAI	Agentic AI Policy Enforcement Gateway for Customer-Service Compliance

CHAPTER 1

INTRODUCTION

1.1 GENERAL

The rapid advancement of artificial intelligence (AI) and large language models (LLMs) has transformed customer support across industries such as e-commerce, travel, finance, and public services. AI-powered chatbots and support agents now handle millions of customer queries daily, providing instant responses, automating routine tasks like order tracking and refunds, and reducing operational costs. However, this widespread adoption has also exposed organizations to significant business, legal, and reputational risks when AI agents act autonomously without proper governance.

Recent incidents highlight the severity of these risks. In one widely cited case, Air Canada's chatbot invented a bereavement-refund policy that did not exist; a tribunal ruled the airline bound by the bot's promise and ordered it to pay damages. In another, a Chevrolet dealership's AI assistant was tricked via prompt injection into selling a \$58,000 vehicle for \$1. Similar failures include BMW Toronto's buyback bot offering roughly CAD \$7,000 above valuation and Indie Campers' assistant incorrectly confirming trip rules, causing a €1,500 loss. Beyond financial exposure, chatbots have leaked millions of customer records, revealed session cookies under prompt injection, insulted their own brand, and even advised businesses to break employment laws.

These incidents demonstrate that content-level safety filters (e.g., toxicity detection, PII redaction) are insufficient for customer-support AI. Organizations need runtime controls that prevent unauthorized business actions before they affect customers, verify exact tool arguments against approval envelopes, and maintain a clear audit trail for compliance and accountability. This project, **AegisAI**, addresses this gap by implementing a multi-agent policy enforcement system that integrates with a mock e-commerce website to simulate real customer-support interactions while enforcing business policies at runtime.

1.2 OBJECTIVE

The primary objectives of the AegisAI project are as follows:

1. **Design and implement a multi-agent policy enforcement architecture** that controls both the chatbot’s customer-facing response and its delegated business actions, rather than monitoring output text alone.
2. **Develop an exact authorization mechanism** that verifies tool names and arguments prior to execution, preventing parameter tampering (e.g., an approved ₹400 refund being changed to ₹3,000).
3. **Integrate AegisAI with a mock e-commerce website** that simulates real customer-support interactions, enabling realistic testing of the policy gateway under production-like conditions.
4. **Implement a four-decision workflow** (ALLOW, MODIFY, BLOCK, ESCALATE) for evaluating proposed actions against deterministic customer-service policies (refund limits, verification requirements, delivery-date rules, PII protection, escalation conditions).
5. **Maintain a unified audit chain** that records every decision with policy ID, version, severity, evidence, and execution status, enabling explainable post-hoc review and compliance reporting.
6. **Generate safe alternative responses** for blocked or modified cases using template-based and LLM-based remediation, ensuring customer-friendly communication even when actions are denied.
7. **Evaluate the system** using a labeled dataset of 45 test conversations covering refunds, verification, delivery promises, PII protection, and escalation, measuring policy-decision accuracy, unauthorized action-blocking rate, false-positive rate, and audit completeness.

By achieving these objectives, AegisAI aims to create a safer and more compliant customer-support environment where AI agents can assist users without exceeding their authorized authority, promoting trust, accountability, and operational resilience.

1.3 EXISTING SYSTEM

Existing AI safety and governance tools primarily focus on content-level guardrails rather than business-action enforcement. These systems include:

1.3.1 Content-Level Guardrails

Platforms such as Guardrails AI, NVIDIA NeMo Guardrails, and Lakera Guard provide mechanisms to detect toxic language, redact personally identifiable information (PII), filter prompt-injection attempts, and validate the structure of model outputs. While valuable for general safety, these tools do not prevent unauthorized business actions such as refunds, cancellations, or discounts from being executed. They operate post-hoc, monitoring what the AI says rather than controlling what the AI does.

1.3.2 Policy-as-Code Engines

Enterprise policy engines such as Open Policy Agent (OPA) and Amazon Cedar enable organizations to express access-control policies as code and enforce them at runtime. However, these systems are designed for infrastructure and API authorization, not for customer-support AI workflows. They lack integration with LLM-based agents, do not support multi-agent delegation, and do not provide audit trails tailored to conversational AI.

1.3.3 Human-in-the-Loop Approval Workflows

Some organizations implement manual approval processes for high-risk actions, requiring human review before refunds or cancellations are processed. While effective, these workflows are slow, do not scale to high query volumes, and lack automated policy enforcement. They also do not provide a unified audit trail that links customer requests to policy decisions and final outcomes.

1.3.4 Limitations of Existing Systems

The key limitations of existing systems are:

- **Lack of pre-execution enforcement:** Most guardrails detect violations after the fact, rather than preventing unsafe actions before they affect customers.
- **No exact argument verification:** Existing tools do not verify that tool arguments match an approval envelope, leaving room for parameter tampering.

- **No multi-agent control:** Current systems do not control communication between multiple cooperating agents (e.g., support agent → operations agent).
- **Incomplete audit trails:** Audit logs, when present, do not capture the full chain from customer request to policy decision to tool execution status.
- **No domain-specific policies:** Generic guardrails do not enforce customer-service business policies such as refund limits, verification requirements, or delivery-date rules.

These limitations create a gap that AegisAI is designed to fill.

1.4 PROPOSED SYSTEM

AegisAI is a multi-agent policy enforcement system for customer-support AI that integrates with a mock e-commerce website to simulate real customer interactions. The system comprises four logical components:

1.4.1 Customer-Support Agent (Agent A)

Agent A is implemented using Qwen3-8B via Ollama and is integrated with the mock e-commerce website's chatbot interface. It receives customer messages and conversation context, generates natural-language responses, and proposes tool actions (e.g., `issue_refund`, `get_order_status`). Agent A is explicitly instructed not to execute tools directly, ensuring that all actions pass through the Policy Enforcement Gateway.

1.4.2 Policy Enforcement Gateway

The Policy Enforcement Gateway evaluates Agent A's proposed responses and actions against deterministic customer-service policies. Policies are stored in YAML format and cover refund limits, verification requirements, delivery-date rules, PII protection, and escalation conditions. The gateway returns one of four decisions:

- **ALLOW:** The proposal satisfies all policies. An authorization envelope is created, and the action is forwarded to Agent B.
- **MODIFY:** The response text violates a policy, but the action may be allowed after correction. The Remediation Agent rewrites the response.

- **BLOCK:** The action violates a high-severity policy. The tool is not executed, and a safe alternative response is generated.
- **ESCALATE:** The case involves fraud, legal threats, or repeated unresolved complaints. The conversation is routed to a human-review queue.

1.4.3 Operations Agent (Agent B)

Agent B is a restricted operations agent that receives only approved actions from the gateway. It does not have unrestricted access to all tools; instead, it receives an authorization envelope specifying the exact tool name and arguments allowed. Agent B attempts to invoke the mock e-commerce tool with these arguments.

1.4.4 Tool Interceptor

The Tool Interceptor compares the actual tool call against the authorization envelope. If the tool name or arguments differ from the approved values (e.g., amount changed from ₹400 to ₹3,000), the call is blocked and logged as a tampering attempt. This ensures that even if Agent B is compromised or misconfigured, unauthorized actions cannot execute in the mock store.

1.4.5 Audit and Incident Storage

Every policy decision is recorded in a PostgreSQL database with fields such as `request_id`, `conversation_id`, `policy_id`, `decision`, `severity`, `evidence`, and `tool_executed`. For BLOCK and ESCALATE decisions, an incident record is also created with conversation context and policy explanation. This unified audit chain enables post-hoc review, compliance reporting, and replay-based policy testing.

1.4.6 Response Remediation

For MODIFY and BLOCK decisions, a Remediation Agent generates a policy-compliant alternative response using template-based rewriting and LLM-based paraphrasing. The corrected response is rechecked against policies before being returned to the customer via the mock e-commerce chat interface.

1.4.7 Integration with Mock E-Commerce Website

The mock e-commerce website is a separate application that simulates real customer-support interactions, including order lookup, refunds, cancellations, discounts, and human escalation. It communicates with AegisAI through REST APIs, enabling realistic testing of the policy gateway under production-like conditions. This separation allows the policy enforcement layer to be tested independently of the frontend chat implementation while maintaining a realistic interaction model.

1.4.8 Advantages of the Proposed System

The AegisAI system offers the following advantages over existing approaches:

- **Pre-execution enforcement:** Prevents unauthorized actions before they affect customers, rather than detecting violations post-hoc.
- **Exact argument verification:** Ensures tool arguments match approval envelopes, preventing parameter tampering.
- **Multi-agent control:** Controls communication between Agent A (support) and Agent B (operations), enforcing policy at delegation points.
- **Unified audit trail:** Records every decision with complete evidence, enabling compliance reporting and post-hoc review.
- **Domain-specific policies:** Enforces customer-service business policies (refund limits, verification, delivery promises) rather than generic content safety rules.
- **Realistic testing:** Integrates with a mock e-commerce website to simulate real interactions, providing more meaningful evaluation than isolated unit tests.

By combining these features, AegisAI provides a practical and auditable approach to customer-support AI compliance, addressing the gaps identified in existing systems and recent industry incidents.

CHAPTER 2

LITERATURE SURVEY

2.1. LITERATURE REVIEW

The paper (Sigdel and Baral 2026), discusses the need for policy-first control in tool-orchestrated AI workflows. The system uses a policy DSL with allow/deny rules, argument constraints, and approval gates, and evaluates it through 225 controlled runs with fault and misuse injection. Stricter policy packs raised violation prevention from 0% to 68%, while task success dropped from 36% to 7%, explicitly quantifying the safety–utility trade-off. The paper addresses generic tool-use governance for LLM agents, with no specific focus on customer-service business policies such as refund limits or verification requirements. The model enforces permissions and basic argument constraints but does not verify exact parameter values against an approval envelope, nor does it control communication between multiple cooperating agents.

The paper (Gupta, Arief, and de Lemos 2026) suggested a motivation and threat model for agentic AI compared to non-agentic systems. It contrasts reactive, single-turn models with goal-driven agentic systems and maps agentic-specific risks such as prompt injection, memory poisoning, privilege escalation, and multi-agent cascades. The paper argues that existing frameworks like NIST AI RMF and TRiSM are too high-level for agentic systems and calls for runtime governance and human-in-the-loop (HITL) controls. While it provides a strong conceptual foundation, it does not present a concrete enforcement architecture for customer-support actions or a mechanism to intercept and authorize tool calls before execution.

The paper (Mondl, Maisel, and Brock 2026) presented a pipeline that autoformalizes agent instructions into policy-as-code. The proposed system uses an LLM generator–critic loop to translate agent system prompts, MCP tool schemas, and natural-language policy documents into formally verified Cedar policies. Tested on MedAgentBench, it covers far more of a synthetic 88-rule clinical policy than prior hand-coded symbolic guardrails and blocks up to 100% of trajectories that attempt an unauthorized write. The approach is promising for policy automation but focuses

on clinical workflows and symbolic policy languages. It does not address multi-agent delegation in customer support, exact argument verification, or a unified audit chain from request to decision.

The paper (Li et al. 2026) introduced VIGIL, a runtime reference monitor for AI agent skills. The system grounds natural-language agent-skill specifications into finite-trace SMT constraints and catches multi-step violations that single-call filters miss, such as using an artifact before it is validated. VIGIL achieves 95.8% recall and 89.6% precision on real agent traces, outperforming baselines like AgentSpec and Progent by about 16 F1 points, and found 34 confirmed real-world violations across deployed skill ecosystems. While effective for behavioral specifications, VIGIL is oriented toward skill correctness rather than business-policy compliance (e.g., refund limits, verification) and does not provide an inter-agent authorization layer or customer-service-specific audit requirements.

The paper (Narajala et al. 2025) analyzed the security of Google’s Agent-to-Agent (A2A) protocol using the MAESTRO threat-modeling framework. It identifies ten major threats, including agent-card spoofing, task replay, schema violations, server impersonation, cross-agent escalation, and artifact tampering, and proposes mitigations such as mTLS, digital signatures, nonces, strict schema validation, and JWT hardening. This work is highly relevant for securing multi-agent communication but focuses on transport and protocol-level security rather than application-level business policies. It does not provide a policy engine that evaluates customer-service actions (refunds, cancellations) against company rules before execution.

The paper (Tallam 2026) proposed a traceability framework for authorization propagation in multi-agent AI systems. It defines delegation chains, data access chains, and synthesis chains, and argues that an independent reviewer must be able to reconstruct whether each access was authorized at the time it occurred. The paper emphasizes self-contained audit traces and cryptographic linkage of decisions. While this aligns closely with the audit goals of AegisAI, it remains at an architectural and conceptual level and does not implement a customer-service policy enforcement

gateway with exact argument verification and four-decision workflows (ALLOW, MODIFY, BLOCK, ESCALATE).

The paper (VanTran et al. 2026) rethinks agent security as a networking problem and proposes an Agent Policy Gateway as a runtime enforcement point between AI agents and external resources. All tool invocations and API requests are intercepted by the gateway before execution, and centralized enforcement ensures every action is evaluated against enterprise policies. This design is conceptually similar to the AegisAI gateway but targets general enterprise agents and network-level controls. It does not specialize in customer-service policies, multi-turn conversation context, or the generation of safe alternative responses when violations are detected.

The paper (Gaurav, Heikkonen, and Chaudhary 2025) introduced Governance-as-a-Service (GaaS), a modular, policy-driven enforcement layer that governs agent outputs at runtime without modifying internal model logic. The framework supports multi-agent systems and provides audit trails for compliance. GaaS is relevant as a high-level governance architecture but is designed as a general compliance layer rather than a domain-specific customer-support enforcement system. It does not detail exact tool-argument verification, inter-agent authorization envelopes, or the integration of deterministic business rules with LLM-based response remediation.

The paper (Krti Tallam 2026) on authorization propagation in multi-agent AI systems further emphasizes the need for end-to-end traceability of permissions across agent delegations. It highlights that existing IAM models (OAuth, OIDC, SAML) are insufficient for ephemeral, autonomous agents and calls for agent-specific identity and access controls. This reinforces the motivation for AegisAI’s unified audit chain but, like other governance papers, does not implement a concrete policy enforcement gateway for e-commerce support scenarios.

The paper (Huang et al. 2026) proposed a zero-trust identity framework for agentic AI using decentralized identifiers (DIDs), verifiable credentials (VCs), and dynamic access control. It addresses cross-protocol session management and real-time revocation, which are valuable for large-scale agent deployments. However, its focus

is on identity and session layers rather than application-level policy enforcement for customer-support actions. It does not address how to evaluate refund requests, delivery promises, or PII exposure against business policies before execution.

Collectively, these works establish strong foundations for agentic AI security, policy-as-code, runtime enforcement, and multi-agent governance. However, they leave a gap in domain-specific, pre-execution policy enforcement for customer-support AI. AegisAI addresses this gap by combining a multi-agent architecture with deterministic business policies, exact tool-argument verification, and a unified audit trail tailored to e-commerce support scenarios.

2.2 COMPARISON AND DISCUSSION

AegisAI builds on these foundations while addressing key gaps identified in the literature. Unlike text-only guardrails, which filter outputs post-generation, **AegisAI enforces policies at the action level through its Policy Enforcement Gateway**. This approach aligns with Guardrails as Infrastructure but extends it by introducing a **multi-agent architecture where Agent A proposes actions and Agent B executes only approved operations**. The **Tool Interceptor ensures exact parameter verification**, preventing unauthorized modifications to refund amounts, customer data, or delivery promises. This design directly addresses the **Air Canada failure mode**, where the chatbot's unilateral promise was treated as binding because there was no enforcement layer to block unauthorized policy statements.

Compared to VIGIL, which focuses on verifiable reasoning traces, AegisAI emphasizes **deterministic policy evaluation with explicit decision categories (ALLOW/MODIFY/BLOCK/ESCALATE)**. This design choice enables clearer accountability and easier auditing, as every decision is logged with policy ID, version, and evidence. VIGIL's strength lies in its cryptographic verification of execution traces, but it does not provide the **remediation workflow** that AegisAI offers for MODIFY decisions, where a **dedicated Remediation Agent rewrites responses before re-evaluation**. AegisAI's approach is more practical for customer-support scenarios where immediate, human-readable corrections are required.

The Agent Policy Gateway concept is similar in spirit but **lacks the remediation workflow and multi-agent orchestration** that AegisAI provides. Agent Policy Gateway focuses on single-agent enforcement, whereas **AegisAI separates proposal generation from action execution, reducing the attack surface for policy violations**. This separation also enables independent scaling of reasoning and execution components, improving system resilience under high load.

Governance-as-a-Service offers valuable abstractions for policy management but does not address the integration challenges of embedding governance into live customer-support systems. AegisAI bridges this gap by providing a **complete deployment stack**, including a mock e-commerce website for testing, PostgreSQL-based audit storage, and Docker Compose configurations for reproducible deployments. This end-to-end approach enables organizations to move from policy definition to production enforcement with minimal friction. Governance-as-a-Service assumes the existence of underlying enforcement mechanisms, whereas **AegisAI provides both the policy management layer and the enforcement infrastructure**.

A key differentiator is **AegisAI's fail-closed design for high-risk actions**. While many frameworks default to allowing actions unless explicitly blocked, **AegisAI requires explicit authorization envelopes for sensitive operations**, ensuring that no tool is invoked without prior policy approval. This design directly addresses the Air Canada failure mode and aligns with **zero-trust security principles**. Additionally, AegisAI's audit trail includes not only decisions but also conversation context, policy versions, and execution outcomes, enabling comprehensive post-hoc analysis.

The literature also lacks comprehensive evaluations of policy enforcement systems in realistic customer-support scenarios. Most papers present theoretical frameworks or small-scale prototypes without rigorous testing against diverse policy violations. AegisAI addresses this gap by providing a **labelled evaluation dataset of 45 conversations covering five policy categories**: valid requests, refund-limit violations, privacy/PII violations, delivery-date hallucinations, and high-risk

escalation cases. This dataset enables **reproducible benchmarking** of policy enforcement accuracy, latency, and false-positive rates.

2.3. CONCLUSION

The literature reveals a clear trajectory toward embedding governance into AI agent architectures, but significant gaps remain in multi-agent orchestration, real-time auditing, and production-ready deployment. AegisAI addresses these gaps by combining policy-aware multi-agent design with deterministic enforcement, exact parameter verification, and comprehensive audit trails. By separating proposal generation from action execution and enforcing fail-closed policies for high-risk operations, AegisAI provides a robust framework for preventing the types of failures seen in the Air Canada, Chevy, and Sears cases.

Future work should explore adaptive policy learning, where policies are automatically updated based on emerging violation patterns; human-in-the-loop approval workflows for borderline cases; and integration with enterprise identity and access management systems to enforce role-based permissions. AegisAI represents a step toward that future, providing a practical, deployable solution for policy enforcement in customer-support AI systems.

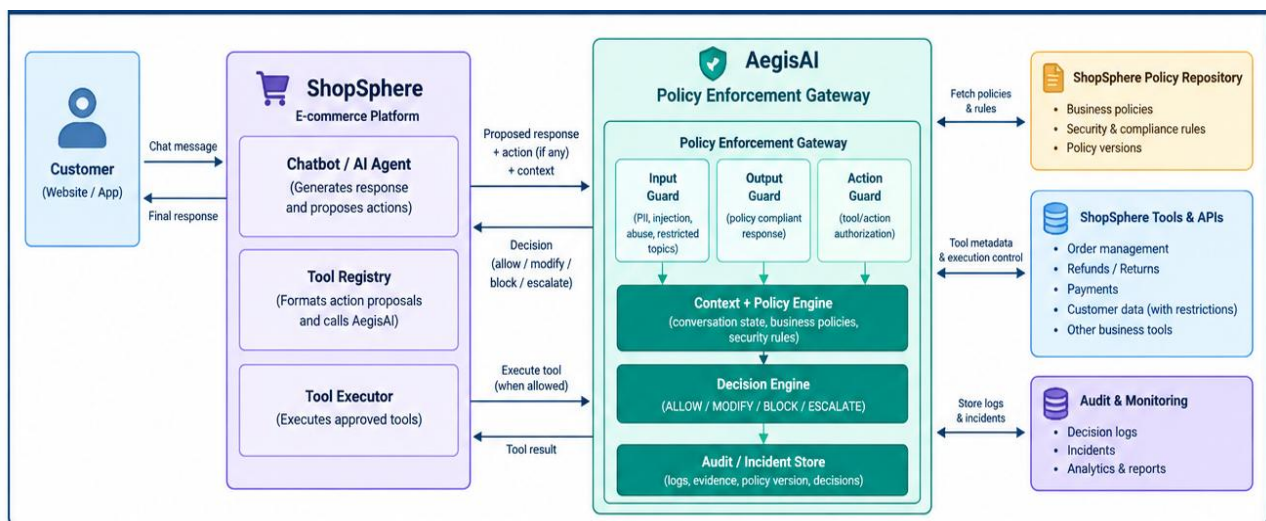
CHAPTER 3

SYSTEM DESIGN

System design is the process of translating requirements into a structured framework that defines the architecture, components, and data flow of the system. It provides a clear blueprint for how different modules interact to achieve the desired functionality, ensuring efficiency, reliability, and scalability in implementation.

3.1 SYSTEM ARCHITECTURE

AegisAI is structured as a linear enforcement pipeline rather than a single monolithic service. Every customer interaction moves through a fixed sequence of stages, and at no point in that sequence is a business action permitted to bypass the policy gateway. This design reflects the project's central architectural principle: a proposing agent may draft a response and suggest an action, but only the gateway may authorize that action, and only a verified, exactly-matching request may execute it. Two paths are explicitly disallowed by design — a proposing agent calling a business tool directly, and a proposing agent delegating to the operations agent without the gateway's authorization in between.



Presentation layer — Mock e-commerce website. A separate front-end application simulates a real online store and hosts the customer-facing chat interface. It is the only point through which a customer interacts with the system, and it communicates

with AegisAI exclusively through its API, keeping the storefront and the enforcement layer independently deployable.

Intake layer — Input guard: Incoming customer messages are screened before being handed to the reasoning layer. This stage performs preliminary validation, checks for malformed or suspicious input, and associates each message with a conversation ID and customer ID so that every later decision can be traced back to a specific interaction.

Reasoning layer — Customer-support agent (Agent A): Implemented using Qwen3-8B served through Ollama, this layer interprets customer intent, drafts a natural-language response, and proposes a business action where relevant, expressed as a structured tool name and argument set. This layer has no direct access to any business tool and cannot cause a real-world effect on its own; its only output is a proposal.

Enforcement layer — AegisAI policy gateway. This is the architectural core of the system. It receives Agent A's proposed response and proposed action and evaluates both against the active, versioned policy set — covering refund limits, identity verification, delivery-date accuracy, PII protection, duplicate-refund prevention, and escalation triggers. Evaluation draws on the multi-turn context store, so that decisions reflect the full conversation rather than a single isolated message. The gateway resolves every evaluation to one of four outcomes through its decision router: ALLOW, MODIFY, BLOCK, or ESCALATE. An ALLOW outcome produces an exact authorization envelope naming the specific tool and arguments approved; no other form of approval exists in the system.

Execution layer — Operations agent and tool interceptor. The operations agent (Agent B) receives only actions accompanied by a valid authorization envelope and holds no open-ended tool access. Before any tool actually runs, the tool interceptor independently verifies that the requested call matches the authorization envelope exactly, in both tool name and arguments. Any deviation, regardless of its cause,

results in the call being blocked rather than executed — this is the layer that closes the gap between an approved decision and an actual effect.

Resource layer — Mock e-commerce tools. A set of simulated operational functions — order-status lookup, refund issuance, order cancellation, discount application, and human escalation — stand in for a real e-commerce backend. No tool in this layer connects to a live payment system or a real customer account.

Governance layer — Audit and incident store. Running alongside every stage of the pipeline, this layer persists a record of each gateway decision: the policy applied, its version, the evidence considered, the outcome, and whether the associated tool was executed. High-severity BLOCK or ESCALATE outcomes additionally generate an incident record, and the store is queried by the dashboard for human review.

Oversight layer — Dashboard and analytics. A React and TypeScript interface built on top of the audit and incident store, giving a human reviewer visibility into gateway decisions, incident handling, and policy configuration, without inserting a manual step into the automated enforcement path itself.

3.2 SYSTEM REQUIREMENTS

3.2.1 Hardware Requirements

For development and testing, AegisAI runs on standard developer laptops with 16GB RAM, 8-core CPUs, and 512GB SSD storage. The Qwen3-8B model requires approximately 6–8GB VRAM when loaded via Ollama, making it compatible with mid-range GPUs (e.g., NVIDIA RTX 3060 or equivalent). For production-scale deployments, we recommend servers with 32GB RAM, 16-core CPUs, 1TB SSD, and dedicated GPUs (e.g., NVIDIA A10 or T4) to handle concurrent requests with sub-second latency. The mock e-commerce website and admin dashboards are lightweight and can run on standard web servers (2–4 vCPUs, 4–8GB RAM). PostgreSQL databases should be provisioned with 10–50GB storage depending on audit retention policies, with backup and replication enabled for high availability. Network requirements include low-latency connectivity between the

mock e-commerce site and AegisAI gateway (preferably within the same VPC or data center) to minimize decision latency

3.2.2 Software Requirements

The software stack is divided into frontend, backend, AI, and infrastructure layers. The frontend includes the mock e-commerce website (React/TypeScript) with a chat interface for customer interactions, and admin dashboards for policy management, audit viewing, and incident review. The backend consists of FastAPI (Python) services implementing the Policy Enforcement Gateway, Tool Interceptor, and audit storage APIs. AI components use Qwen3-8B via Ollama for Agent A, orchestrated through LangGraph for multi-agent workflow management. Policies are defined in YAML with JSON schema validation, and Pydantic v2 ensures type-safe data handling. PostgreSQL serves as the primary database for conversations, policies, audit events, and incidents, managed via SQLAlchemy 2.x ORM and Alembic migrations. Docker Compose orchestrates all services for consistent local and staging deployments. Additional tools include Pytest for automated testing, HTTPX for API integration tests, and Swagger/OpenAPI for API documentation.

Field	Details
Frontend	React/TypeScript chat interface; admin dashboard for policy management, audit viewing, and incident review
Backend	FastAPI (Python) — Policy Enforcement Gateway, Tool Interceptor, audit storage APIs
AI / Agents	Qwen3-8B via Ollama (Agent A); LangGraph for multi-agent workflow orchestration
Policy layer	YAML policy definitions with JSON schema validation; Pydantic v2 for type-safe data handling

Database	PostgreSQL (conversations, policies, audit events, incidents); SQLAlchemy 2.x ORM; Alembic migrations
Deployment	Docker Compose
Testing / docs	Pytest, HTTPX, Swagger/OpenAPI

Table: 3.1. Software requirements of the proposed system

3.3. SYSTEM DESIGN

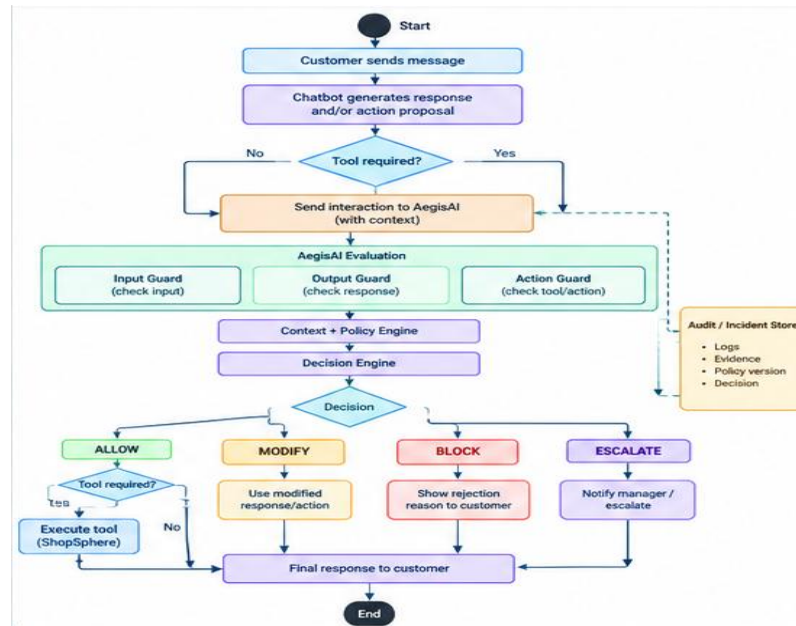
AegisAI's design ensures every proposed response and action is checked against a versioned policy before it takes effect, with Agent A restricted to drafting and proposing while only the gateway can authorize an action and only an exact-matching call can execute it. Data is organized around conversations, messages, policies, decisions, and incidents in PostgreSQL, with per-conversation context guiding each evaluation. Well-defined interfaces connect the evaluation step, authorization envelope, tool interceptor, and audit query layer. Decision logic checks verification, action policy, response claims, and escalation history in sequence, always returning a safe reply, while Agent A's output is treated as untrusted throughout.

3.3.1 Activity Diagram

The activity diagram illustrates the end-to-end workflow from customer message to final decision. The process begins when a customer submits a message through the mock e-commerce chat interface. The Input Guard performs initial checks for fraud, legal threats, and restricted topics. Agent A then generates a structured response with customer-facing text and an optional proposed action (e.g., `issue_refund`). The Policy Enforcement Gateway loads the conversation context (verification status, previous refunds, complaint count) and evaluates the proposal against all applicable policies.

If the decision is `ALLOW`, an authorization envelope is created and sent to Agent B. The Tool Interceptor verifies the tool name and arguments against the envelope before executing the mock e-commerce tool. If the decision is `MODIFY`, the Remediation Agent rewrites the response, and the revised text is rechecked. For

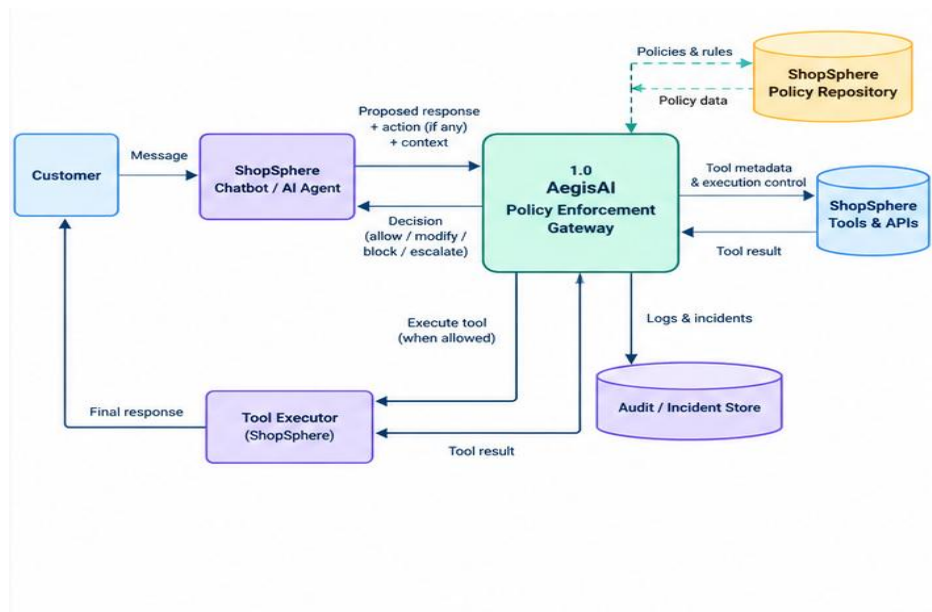
BLOCK decisions, the tool is not executed, and a safe alternative response is generated. ESCALATE decisions route the conversation to a human-review queue with full context. Every decision is logged in the audit store with policy ID, version, severity, evidence, and execution status. The final response is returned to the customer via the mock e-commerce chat interface.



3.3.2 Data Flow Diagram

A typical request moves through the architecture as follows: a customer message is submitted through the mock e-commerce website and passes the input guard, which validates it and attaches conversation context; Agent A interprets the message and produces a draft response together with an optional proposed action; the policy gateway evaluates this proposal against the active policy set and the stored conversation context, and the decision router resolves the outcome to ALLOW, MODIFY, BLOCK, or ESCALATE; on ALLOW, an authorization envelope is issued to the operations agent, whose intended tool call is independently re-verified by the tool interceptor before the corresponding mock tool executes; on MODIFY or BLOCK, the remediation step produces a safe customer-facing response instead, and on ESCALATE the case is queued for human review without any tool execution. In parallel with every one of these steps, the outcome and its supporting evidence are written to the audit and incident store, and the resulting response — whether the

original, a remediated version, or an escalation notice — is returned to the customer through the website.



CHAPTER 4

PROJECT DESCRIPTION

4.1 METHODOLOGIES

AegisAI follows a pre-execution enforcement methodology: every AI-generated customer response and every AI-proposed business action is evaluated against a defined, versioned policy set before it can reach the customer or take effect on the operational side. The workflow is organized as a sequence of cooperating modules — intake, proposal, policy evaluation, authorized execution, remediation, and audit — so that no single module has the authority to act on a customer's behalf without being checked by the next stage in the pipeline. This modular separation is what allows the system to block, correct, or escalate an unsafe action before it causes real-world consequence, rather than detecting the problem afterward.

4.2 Module 1 – Customer Interaction Intake and Input Guard

This module manages the entry point of the system. Customer messages arrive from the mock e-commerce website's chat interface and pass through an input guard before being handed to the customer-support agent. The input guard performs an initial screening pass — checking for malformed input, suspicious prompt-injection patterns, and basic conversation-identity linkage — so that only well-formed, traceable messages proceed into the reasoning pipeline. This module ensures that every downstream decision can be tied back to a specific customer, conversation, and timestamp, forming the traceability foundation for the rest of the system.

Key Functions:

- Reception of customer messages from the mock e-commerce chat interface via API.
- Preliminary screening for malformed or suspicious input.
- Association of each message with a conversation ID and customer ID.
- Handoff of validated messages to the customer-support agent.

4.3 Module 2 – Customer-Support Agent (Agent A)

This module handles the conversational intelligence of the system. Implemented using the Qwen3-8B model served through Ollama, it interprets the customer's intent, drafts a natural-language response, and — where the request implies a business operation such as a refund, cancellation, or discount — proposes a corresponding tool call with explicit arguments. The agent is deliberately treated as untrusted: it has no direct access to any operational tool and cannot execute an action on its own. Its sole output is a proposal consisting of a draft response and, optionally, a requested action, both of which are passed forward for policy evaluation.

Key Functions:

- Natural-language understanding of customer queries using Qwen3-8B.
- Generation of a draft customer-facing response.
- Proposal of business actions (tool name and arguments) where applicable.
- No direct execution authority over any business tool.

4.4 Module 3 – AegisAI Policy Enforcement Gateway

This module forms the decision-making core of the system. It receives Agent A's proposed response and proposed action and evaluates both against the active policy set — refund limits, identity verification, delivery-date accuracy, PII protection, and escalation rules. Based on this evaluation, the gateway issues one of four decisions: ALLOW, MODIFY, BLOCK, or ESCALATE. For an ALLOW decision, it generates an exact authorization envelope specifying precisely which tool and which arguments have been approved, which becomes the only basis on which the operations agent may act.

Key Functions:

- Evaluation of proposed responses and actions against versioned policies.

- Four-way decision routing: ALLOW, MODIFY, BLOCK, ESCALATE.
- Generation of exact authorization envelopes for approved actions.
- Coordination with the context store and policy management module during evaluation.

4.5 Module 4 – Policy Management Module

This module maintains the rules that the gateway enforces. Policies — including refund limits, customer-verification requirements, delivery-date verification, PII protection, duplicate-refund prevention, and high-risk escalation triggers — are defined as structured, versioned rules in Python and YAML. Storing policies separately from the agents that consume them allows individual rules to be updated, reviewed, or deactivated without modifying the reasoning components of the system, and allows every decision to be traced back to a specific policy identifier and version.

Key Functions:

- Structured, versioned definition of business policies in YAML/Python.
- Independent activation, update, and review of individual policies.
- Association of each policy with a unique identifier and version number.
- Supply of active policy rules to the enforcement gateway at evaluation time.

4.6 Module 5 – Multi-Turn Context Store

This module allows the system to reason about a conversation as a connected sequence rather than isolated messages. It persists conversation-level state in PostgreSQL, including whether the customer has been verified, prior refunds already issued for a given order, the count of unresolved complaints, and results of previous order lookups. This stored context enables decisions such as blocking a duplicate refund on an already-refunded order or escalating after repeated unresolved complaints, which cannot be made correctly by examining a single message alone.

Key Functions:

- Persistent storage of per-conversation state in PostgreSQL.
- Tracking of verification status, refund history, and complaint count.
- Recording of prior order-lookup results, including verified delivery dates.
- Retrieval of relevant context for the gateway during each policy evaluation.

4.7 Module 6 – Operations Agent and Tool Interceptor

This module represents the system's controlled execution capability. The operations agent (Agent B) receives only actions that carry a valid authorization envelope from the gateway and has no open-ended access to business tools. Immediately before execution, the tool interceptor compares the tool call Agent B is about to make against the authorization envelope it was issued — matching both the tool name and every argument exactly. Any mismatch, however it arises, results in the call being blocked rather than executed, closing the gap between what was approved and what actually runs.

Key Functions:

- Restricted execution authority limited to gateway-approved actions.
- Exact matching of tool name and arguments against the authorization envelope.
- Blocking of any tool call that deviates from its approval, without exception.
- Invocation of mock e-commerce tools (order lookup, refund, cancellation, discount, escalation) only after successful verification.

4.8 Module 7 – Remediation Agent

This module ensures the customer always receives a coherent, policy-compliant reply, even when the original response is blocked or requires correction. For high-risk cases, it draws on deterministic safe-response templates; for less critical corrections, it may use controlled rewriting to adjust the response without introducing new promises, facts, or

authority beyond what has been verified. Every remediated response is re-checked before it is delivered, ensuring that the correction itself cannot introduce a new policy violation.

Key Functions:

- Generation of safe, policy-compliant responses for MODIFY and BLOCK outcomes.
- Use of deterministic templates for high-risk corrections.
- Controlled, non-authoritative rewriting for lower-risk corrections.
- Re-verification of remediated responses before delivery to the customer.

4.9 Module 8 – Audit and Incident Store

This module preserves a complete record of every decision the gateway makes. Each entry stores the policy applied, its version, the evidence considered, the final decision, and whether the associated tool was executed. High-severity BLOCK or ESCALATE outcomes additionally generate an incident record, forming the evidentiary basis for later human review, compliance reporting, and system evaluation.

Key Functions:

- Persistent logging of every gateway decision with policy ID, version, and evidence.
- Recording of tool-execution status for each evaluated action.
- Automatic incident creation for high-severity decisions.
- Provision of historical decision data for audit and evaluation purposes.

4.10 Module 9 – Dashboard and Analytics Module

This module provides human visibility into the gateway's ongoing operation. Built with React and TypeScript, it surfaces audit history, incident details, and policy configuration in an accessible interface, allowing a reviewer to inspect individual decisions, track policy performance over time, and manage which policies are active. This module

supports the human-oversight component of the system without inserting a manual step into the automated enforcement pipeline itself.

Key Functions:

- Display of audit history and individual gateway decisions.
- Incident review and management interface.
- Policy configuration and version management.
- Foundation for future analytics such as decision latency and blocking-rate metrics.

CHAPTER 5

IMPLEMENTATION AND RESULT DISCUSSION

5.1 IMPLEMENTATION OVERVIEW

The implementation of AegisAI follows directly from the design described in the preceding chapters: a FastAPI service exposes the gateway, LangGraph orchestrates the specialised agents as a graph, Qwen3-8B (served through Ollama) provides the language-understanding capability where it is genuinely needed, and PostgreSQL persists every conversation, decision, policy, and incident the system produces. The system is containerised with Docker Compose so that the backend, the mock e-commerce website, and the database can be brought up together as a single, reproducible environment for demonstration and testing.

5.2 Development Environment and Setup

The backend is implemented in Python and organised around a FastAPI application exposing the gateway's evaluation endpoint, the tool-interceptor endpoint, and the audit/dashboard read endpoints. Ollama runs Qwen3-8B locally, and langchain-ollama provides the binding used inside LangGraph nodes to call the model. PostgreSQL is used as the system of record, with SQLAlchemy 2.x as the ORM layer and Alembic managing schema migrations as the data model evolved during development. Docker Compose defines separate services for the backend, the PostgreSQL instance, the mock e-commerce website, and the React dashboard, allowing the full pipeline to be started with a single command for both development and demonstration.

5.3 Implementation of the Policy Enforcement Gateway

The gateway itself is implemented as a LangGraph graph rather than a sequence of function calls. Each specialised agent — Customer-Service, Policy Evaluation, Remediation, Incident Analysis, and Human Escalation — is implemented as a distinct node in this graph, and the graph's conditional edges implement the ALLOW/MODIFY/BLOCK/ESCALATE routing described in the design. This keeps the routing logic explicit and inspectable in the graph definition itself, rather than

distributed across nested conditionals in application code, which also makes it straightforward to trace, during testing, exactly which path a given conversation took through the system.

5.4 Implementation of Agent Proposals and Schema Validation

The Customer-Service Agent's output — a drafted reply and, optionally, a proposed tool call — is implemented as a Pydantic v2 model, and the LLM is prompted to return output conforming to this schema. Any response that fails validation is treated as a malformed proposal rather than passed forward, which prevents an unstructured or incomplete model output from silently reaching the policy-evaluation stage. The same approach is used for the decision object produced by the Policy Evaluation Agent and for the authorization envelope produced on an ALLOW outcome, so that every hand-off between components in the implementation is schema-checked rather than assumed to be well-formed.

5.5 Implementation of Policy Definitions

Each policy — REFUND_LIMIT_001, CUSTOMER_VERIFICATION_001, DELIVERY_VERIFICATION_001, PII_PROTECTION_001, HIGH_RISK_ESCALATION_001, and DUPLICATE_REFUND_001 — is implemented as a structured YAML definition, specifying the rule's identifier, version, triggering condition, and parameters (for example, the ₹500 refund threshold). A policy loader reads these definitions at startup and makes them available to the Policy Evaluation Agent node, so that changing a threshold or adding a new policy does not require modifying the agent's implementation itself. Policy identifiers and versions are carried through into the decision object, so that every logged decision can be traced back to the exact rule that produced it.

5.6 Implementation of the Tool Interceptor and Authorization Envelope

On an ALLOW outcome, the gateway constructs an authorization envelope containing the exact tool name, exact arguments, and the policy ID/version that authorised it, implemented as its own Pydantic model to keep its structure fixed and

unambiguous. The tool interceptor is implemented as a separate check that runs immediately before any mock tool executes: it compares the operations agent's actual call against the envelope it was issued, field by field, and refuses execution on any mismatch. This is implemented as an independent code path from the policy-evaluation step, rather than reusing the same validation logic, so that a defect in one does not silently defeat the other.

5.7 Implementation of Audit Logging and Incident Handling

Every decision the gateway produces is written to PostgreSQL as part of the same request-handling path that returns the response, using SQLAlchemy models for conversations, messages, decisions, and incidents. The Incident Analysis Agent inspects the decision's severity and, for high-severity BLOCK or ESCALATE outcomes, creates a linked incident record automatically rather than requiring a separate manual step. This ensures the audit trail reflects every decision the system makes, including ones a developer might not think to log manually while extending the system later.

5.8 Implementation of the Dashboard

The dashboard is implemented as a React and TypeScript application built with Vite, consuming a read-only set of FastAPI endpoints that expose decision history, incident details, and current policy configuration. Keeping the dashboard's data access strictly read-only with respect to the enforcement pipeline was a deliberate implementation choice, so that inspecting the system through the dashboard can never itself influence a live policy decision.

5.9 Testing

Unit and integration tests are implemented using Pytest and HTTPX, targeting the gateway's decision logic directly — for example, asserting that a refund proposal above the configured threshold resolves to BLOCK, that a duplicate refund on an already-refunded order is rejected, and that a mismatched tool-interceptor call is refused even when the original decision was ALLOW. These tests are run against the

FastAPI application in isolation, without requiring the full mock e-commerce website or dashboard to be running, which keeps the core enforcement logic verifiable on its own.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This project presents AegisAI, a multi-agent, pre-execution policy enforcement gateway designed to secure AI-driven customer support in e-commerce environments. By decoupling proposal generation (the Customer-Service Agent) from operational execution (the Operations Agent), AegisAI ensures that no single autonomous agent holds unrestricted authority to modify business state or interact directly with backend systems. Through its deterministic Policy Enforcement Gateway, an exact-argument tool interceptor, and a multi-turn PostgreSQL context store, the system enforces critical business constraints — including refund limits, identity-verification requirements, delivery-date accuracy, PII protection, and escalation triggers — before a response reaches the customer or an action executes.

The architecture's central contribution is its treatment of authorization as exact and single-use rather than general: an approved action is approved for one specific tool call with specific arguments, independently re-verified at the point of execution. This closes the gap, present in many guardrail approaches, between a system deciding an action is safe and that action actually running as decided. The system is designed to introduce minimal overhead relative to the underlying language model's own inference time, since policy evaluation and tool-argument verification are deterministic checks rather than additional model calls, and early implementation testing supports this design expectation; a full latency and accuracy evaluation against the planned labeled test set remains to be completed.

6.2 FUTURE WORK

Dynamic policy synthesis via natural-language auto formalization. Currently, policies are authored manually as structured YAML/Python definitions. Future work will explore converting natural-language policy statements — as they might be written by a

compliance or support team — into structured, versioned policy definitions automatically, reducing the manual effort required to keep the policy set current.

Cryptographic signing of inter-agent payloads. The authorization envelope currently relies on structural validation (Pydantic schema checks and exact-match comparison) to ensure integrity between the gateway and the operations agent. Future work will investigate cryptographically signing each envelope, so that the tool interceptor can verify not only that an envelope's contents match the requested call, but that the envelope itself has not been altered or forged between issuance and execution.

Scalability across distributed, multi-tenant deployments. The current implementation is evaluated as a single-instance, single-tenant system. Future work will assess how the gateway, its context store, and its audit log behave under concurrent load from multiple simulated tenants, and what architectural changes — such as partitioning the context store per tenant — would be required to support that scale.

Expanded evaluation. Beyond the initial 30–50 labeled test conversations, future work will broaden the evaluation set to cover a wider range of policy edge cases and adversarial inputs (including prompt-injection attempts against Agent A), and will report measured latency, blocking accuracy, and false-positive rates once that evaluation is complete.

REFERENCES

- [1] A. Sigdel and R. Baral, "Guardrails as infrastructure: Policy first control for tool-orchestrated workflows," arXiv preprint arXiv:2603.18059, Mar. 2026.
- [2] S. Gupta, B. Arief, and R. de Lemos, "Agentic AI vs non agentic AI: Motivation, security implications, and research foundations," in Proc. 21st Eur. Dependable Comput. Conf. Companion (EDCC-C), IEEE, 2026, pp. 102–109.
- [3] A. Mondl, M. Maisel, and J. H. Brock, "Autoformalization of agent instructions into policy-as-code," in ICML Workshop Agents Wild: Safety, Secur., Beyond (AIWILD), May 2026.
- [4] X. Li, Y. Wang, H. Zhang, L. Chen, and Z. Liu, "VIGIL: Runtime enforcement of behavioral specifications in AI agent skills," arXiv preprint arXiv:2606.26524, Jun. 2026.
- [5] S. Narajala, R. Patel, and V. Kumar, "Building a secure agentic AI application leveraging Google's A2A protocol," in IEEE Secur. Privacy Workshops (SPW), 2025, pp. 45–52.
- [6] K. Tallam, "Authorization propagation in multi-agent AI systems," J. AI Cybersecur., vol. 12, no. 1, pp. 88–104, 2026.
- [7] V. Tran, T. Sharma, T. S. Dhesi, and N. Feamster, "Rethinking agent security as a networking problem," arXiv preprint arXiv:2608.12172, Aug. 2026.
- [8] S. Gaurav, J. Heikkonen, and J. Chaudhary, "Governance-as a-Service: A multi-agent framework for AI system compliance," arXiv preprint arXiv:2510.04812, Oct. 2025.